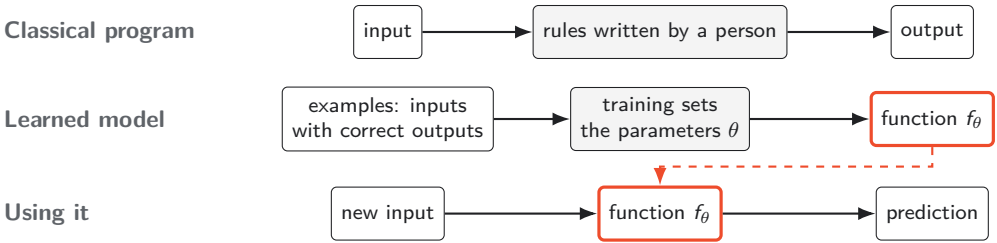
Deep Learning 1

Lecture 1: logistic regression and the multilayer perceptron

David Mojžíšek

model · loss · optimization · evaluation

Deep learning: a function learned from examples



- **Artificial intelligence (AI)**: systems performing tasks associated with intelligence, including programs with hand-written rules (row 1).
- **Machine learning (ML)**: the behaviour is fitted to data (rows 2, 3). Examples: linear and logistic regression, decision trees.
- **Deep learning (DL)**: ML where f_θ is a neural network of several layers, with thousands to billions of parameters. It learns its features from raw input: pixels, text, sound.

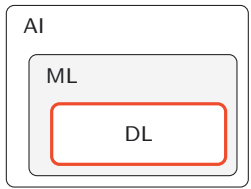
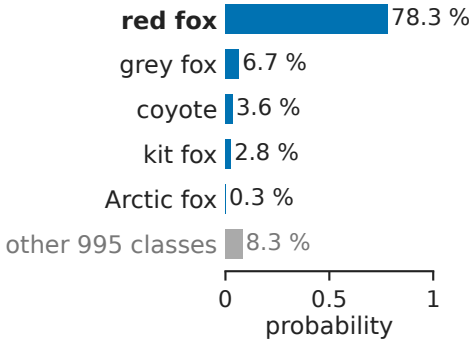


Image classification: a probability for each of 1000 classes





- Input: the pixels. Output: one probability per class; “red fox” gets 78.3 %. Here the next four classes are other foxes and the coyote.
- The network contains no hand-written rules about foxes. It was trained on about 1.3 million labelled photographs (ImageNet).

Model: ConvNeXt-Tiny (torchvision). Photo: Uoaei1, Wikimedia Commons, CC BY-SA 4.0, cropped.

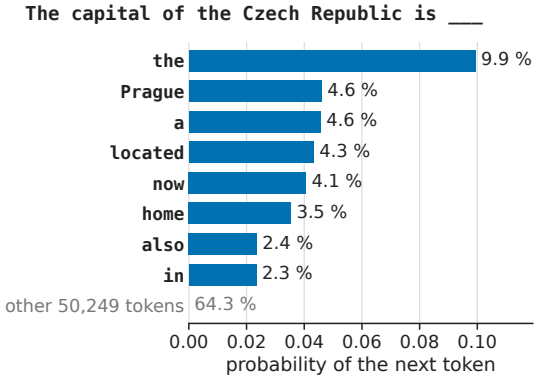
Object detection and instance segmentation



One network, one pass: for every object a box, a class with a score, and a pixel-accurate mask. Found: 1 person, 2 cows, 1 car, 1 motorcycle; all scores ≥ 0.93.

Model: Mask R-CNN (torchvision, COCO weights), threshold 0.7. Photo: Jos1950, Wikimedia Commons, CC BY-SA 4.0, cropped.

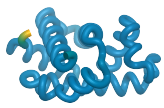
Language modelling: next-token prediction



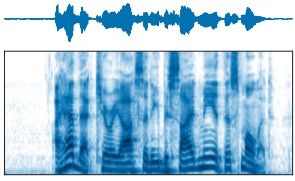
- Input: the text so far.
- Output: a probability for each of 50 257 possible next tokens (a token is a word or a piece of a word).
- A chat answer is this step, repeated.

Computed with GPT-2 small (2019, 124 million parameters); it gives "Prague" only 4.6 %. Current models are sharper.

Common structure: a parametric function trained on an average loss



AlphaFold:
sequence → 3D structure



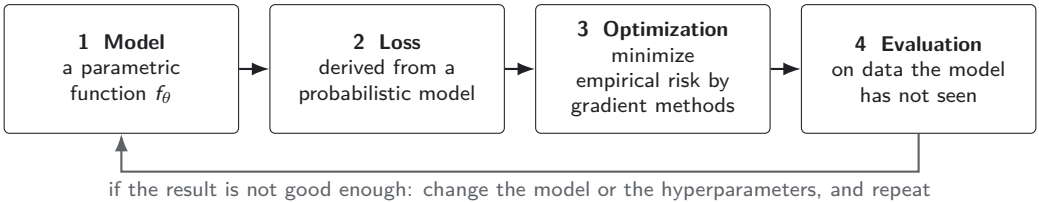
Speech: sound → "I had that curiosity beside me . . . "

	input x	prediction from $f_\theta(x)$
image classifier	pixels	class probabilities
detector	pixels	boxes, classes, masks
language model	text so far	next-token probs.
AlphaFold	amino acids	3D coordinates
speech	spectrogram	text probs.

- Each is a function f_θ with adjustable numbers θ, millions to hundreds of billions of them.
- Each is trained by minimizing an **average loss** over examples.
- For a language model that loss is the cross-entropy of a classification with about 10^5 classes.

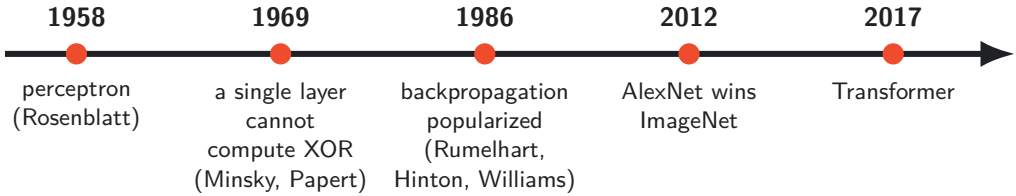
Structure: AlphaFold DB, AF-P69905-F1 (human hemoglobin α), CC BY 4.0. Audio: VOiCES corpus, CC BY 4.0, 3.4 s; the plot is its spectrogram (time × frequency).

Components of deep learning: model, loss, optimization, evaluation

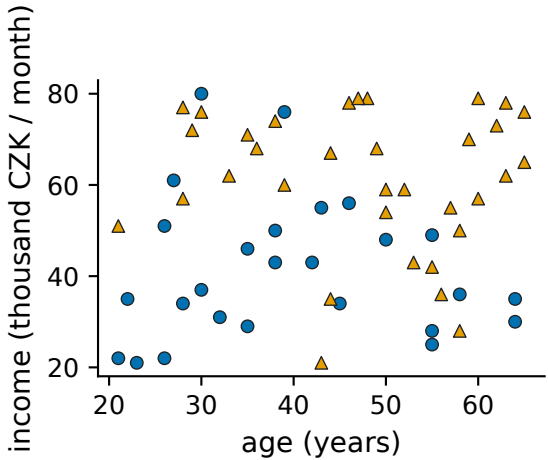


Deep learning = a parametric function f_θ + a loss derived from a probabilistic model + minimization of empirical risk by gradient methods + evaluation on data the model has not seen.

Timeline 1958–2017



Running example: binary classification of customers



- **Feature vector** $x = (\text{age}, \text{income}) \in \mathbb{R}^2$
- **Label** $y \in \{0, 1\}$: ▲ 1 buys, ● 0 does not.
- Predicting a class is classification; predicting the amount spent would be **regression**.

Goal. A line that separates the two classes as well as possible. Many lines look reasonable and none is perfect; "best" needs a definition and a method to find it.

Synthetic data: 75 customers from one random generator. 60 are shown and used for training; 15 are set aside as a *test set* for the evaluation block.

Notation

dataset	$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ with $x_i \in \mathcal{X}$ (here $\mathbb{R}^2$) and $y_i \in \mathcal{Y}$ (here $\{0, 1\}$)
i.i.d. assumption	the pairs (x_i, y_i) are drawn independently from one unknown distribution P
model	a function f_θ of the input x; its output is turned into a prediction of y
parameters θ	the numbers inside f_θ that training sets
model family	$\{f_\theta : \theta \in \Theta\}$, all functions f_θ obtained by varying θ
hyperparameters	what is fixed before training: the family, the learning rate, ...

Training adjusts θ step by step. It never changes the hyperparameters.

Feature scaling (z-score) and data leakage

z-score (standardization) of feature j: $x_j \leftarrow \dfrac{x_j - \mu_j}{s_j}$, μ_j = mean, s_j = standard deviation (with $1/n$) of feature j in the training set.

- Here: age $\mu = 44.2$, $s = 13.2$ years; income $\mu = 52.6$, $s = 18.2$ thousand CZK.
- On the training set every feature then has mean 0 and standard deviation 1. Gradient methods behave better when features have similar ranges.
- The same μ_j, s_j are applied to the test customers and to any future customer. The later customer plots use z-scored axes.

Data leakage. Computing μ_j, s_j on all 75 customers would let the 15 test customers influence training: the mildest form of leakage. General rule: every fitted quantity is fitted on the training set only.

Linear score and decision boundary

Exact

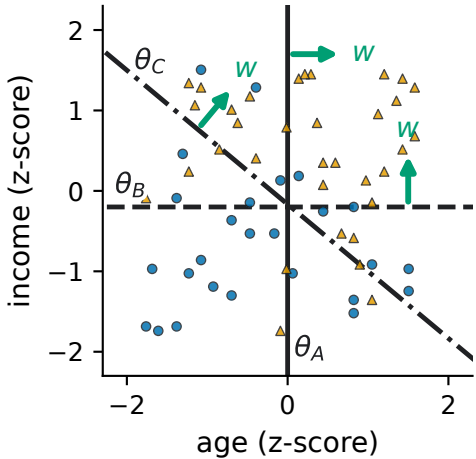
$$z = w^\top x + b = w_1 x_1 + w_2 x_2 + b$$

$w^\top x$: multiply entry by entry, then add.
$\theta = (w, b)$, $w \in \mathbb{R}^2$, $b \in \mathbb{R}$.

- **Decision boundary**: all x with $z = 0$; a line in $\mathbb{R}^2$, a **hyperplane** in $\mathbb{R}^d$. Predict "buys" if $z \geq 0$; z is any real number.

Intuition

- w (green) is the **normal vector**: along the line z stays 0, so w is perpendicular to it. It points to the side where $z > 0$.
- b shifts the line without turning it.



Three hand-picked parameter vectors $\theta_A, \theta_B, \theta_C$.

Sigmoid: from score to probability

Exact

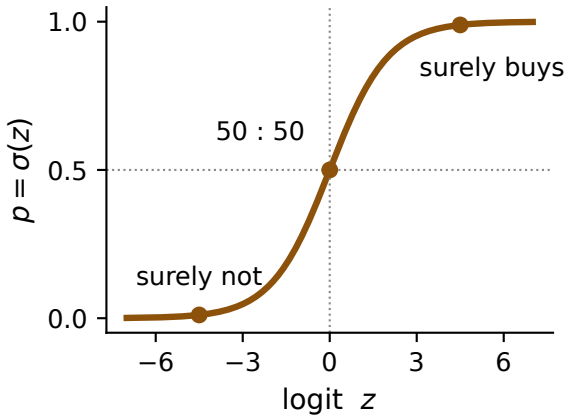
sigmoid (standard logistic function)

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma : \mathbb{R} \to (0, 1)$$

- increasing, $\sigma(0) = \frac{1}{2}$
- $\sigma(-z) = 1 - \sigma(z)$

Far from the boundary the model is confident. On the boundary, $z = 0$, it says $50 : 50$.

Intuition: a curve that squashes any score into $(0, 1)$



Logistic regression as a statistical model

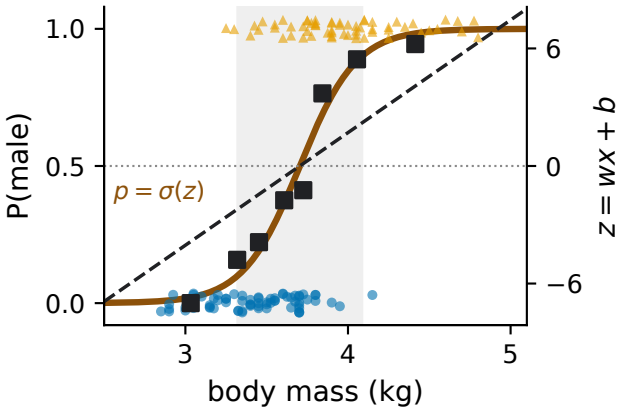
$$p \;=\; P(y=1 \mid x;\,\theta) \;=\; \sigma(w^{\top}x + b), \qquad \text{hence} \qquad P(y=0 \mid x;\,\theta) = 1-p.$$

- The model outputs a **conditional distribution** $p(y \mid x;\theta)$ over the labels: a probability for $y=1$ and for $y=0$.
- Solve $p=\sigma(z)$ for z:

$$p = \frac{1}{1+e^{-z}} \implies e^{-z} = \frac{1-p}{p} \implies z = \log\frac{p}{1-p}.$$

- The ratio $p:(1-p)$ is called the odds ($3:1$ means $p=0.75$). So z is the logarithm of the odds: the **logit** (log-odds).
- The model says: the log-odds of buying are a linear function of the features.

Example: 1D logistic regression, penguin sex from body mass

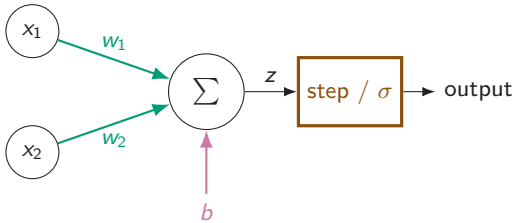


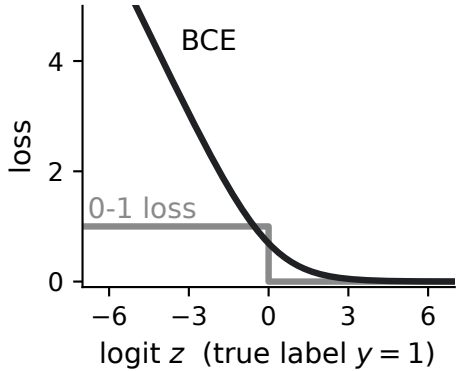
- 128 Adélie and Chinstrap penguins: 60 males (▲ 1), 68 females (● 0). One feature x = body mass in kg.
- Model $z = wx + b$ (dashed line, right axis), $p = \sigma(z)$ (curve). Fitted: $w = 5.7$ per kg, $b = -21.3$; $z = 0$ and $p = 0.5$ at 3.7 kg.
- Black squares: the observed share of males in 8 mass bins. They follow the curve: σ models a *frequency*.

Grey band: 3.3 to 4.1 kg, where $0.1 < p < 0.9$. It holds 65 % of the birds: the classes overlap, and the threshold decides. w, b are fitted (next block) on 128 of 214 birds; 86 are held out.

Data: Palmer penguins (Gorman, Williams, Fraser 2014; Horst, Hill, Gorman 2020), CC0.

Perceptron and the 0-1 loss





- Rosenblatt's **perceptron**: output 1 if $z \geq 0$, else 0. Logistic regression: the same z through σ instead of a step.
- Counting errors cannot be minimized by gradient methods: the **0-1 loss** (grey) is piecewise constant in θ, its derivative is 0 almost everywhere and gives no direction.
- Training needs a loss that is smooth in θ (black, derived next), and a definition of which θ is best.

Risk, empirical risk and empirical risk minimization

A **loss** $\ell(f_\theta(x), y)$ measures how bad the prediction for x is when the truth is y.

Risk (expected loss on new data): $R(\theta) = \mathbb{E}_{(x,y)\sim P}\big[\ell(f_\theta(x), y)\big]$

Empirical risk (average loss on the training set): $\widehat{R}(\theta) = \dfrac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$

Empirical risk minimization (ERM): $\theta^* \in \arg\min_{\theta\in\Theta} \widehat{R}(\theta)$ ($\arg\min$ = the θ where the minimum is attained)

- The goal is a small R, the average loss over all future customers. P is unknown, so R cannot be computed. $\widehat{R}$ is the average over the 60 customers we have.
- "Training" means: find θ that (approximately) minimizes $\widehat{R}$. The difference $R(\theta) - \widehat{R}(\theta)$ is the **generalization gap**.

Adapted from Jentzen et al., arXiv:2310.20360, Introduction, eqs. (3) and (10), where ℓ is the squared error.

Overview of losses: MSE, BCE, CE

loss	per-sample formula	target y	typical task	network output
MSE	$(y-\hat{y})^2$	a real number	regression: price, temperature, amount spent	one number $\hat{y}$
BCE	$-\big[y\log p+(1-y)\log(1-p)\big]$	0 or 1	two classes, yes/no: buys, spam, damage	one logit z, $p=\sigma(z)$
CE	$-\log p_y$	one of K classes	digit, garment, next token	K logits, $p=\mathrm{softmax}(z)$

- The type of the target y determines the loss. The training objective is always the average over the samples, $\widehat{R}$.
- Several independent yes/no labels per sample: one logit and one BCE per label.
- MSE on a probability output trains poorly (appendix). For classes, BCE or CE.
- MSE = mean squared error, BCE = binary cross-entropy, CE = cross-entropy. Each formula is a negative log-likelihood.

Probabilistic model: Bernoulli distribution of the label

- The model says: given x_i, the label is a coin flip with success probability $p_i = \sigma(w^\top x_i + b)$:

$$y_i \mid x_i \;\sim\; \mathrm{Bernoulli}(p_i).$$

- One formula for both outcomes:

$$p(y_i \mid x_i; \theta) \;=\; p_i^{y_i}(1-p_i)^{1-y_i} \;=\; \begin{cases} p_i & \text{if } y_i = 1, \\ 1 - p_i & \text{if } y_i = 0. \end{cases}$$

- This is the probability the model gave to what actually happened. A good θ makes it large for every training example.

Likelihood of the dataset and maximum likelihood estimation

- The samples are independent (i.i.d.), and for independent events probabilities multiply. So the probability of all observed labels, given the inputs $x_1, \ldots, x_n$, is a product:

$$L(\theta) \;=\; \prod_{i=1}^{n} p(y_i \mid x_i; \theta) \;=\; \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

  Full joint: $\prod_i p(x_i, y_i) = \prod_i p(y_i \mid x_i; \theta)\, p(x_i)$. The factors $p(x_i)$ do not contain θ and drop out of the maximization.

- **Likelihood vs. probability**: one formula, read two ways. As a function of the labels with θ fixed it is a probability. As a function of θ with the data fixed it is the (conditional) **likelihood** $L(\theta)$.
- **Maximum likelihood estimation (MLE)**: choose the parameters under which the observed data are most probable, $\theta_{\mathrm{MLE}} \in \arg\max_\theta L(\theta)$.

Log-likelihood: why the logarithm

1. **Products become sums.** $\log(ab) = \log a + \log b$, and sums are easy to differentiate:

$$\log L(\theta) = \sum_{i=1}^{n} \big[\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \,\big] \qquad \text{(the \textbf{log-likelihood}).}$$

2. **Numerical underflow.** A product of many numbers below 1 leaves the floating-point range: $0.5^{1000} = 9.3 \cdot 10^{-302}$, and 0.5^{2000} evaluates to 0 in 64-bit floats. The sum of logs is harmless: $2000 \log 0.5 = -1386.3$.
3. **Same maximizer.** log is strictly increasing, so $L(\theta_1) > L(\theta_2) \iff \log L(\theta_1) > \log L(\theta_2)$, hence

$$\arg\max_{\theta} L(\theta) = \arg\max_{\theta} \log L(\theta).$$

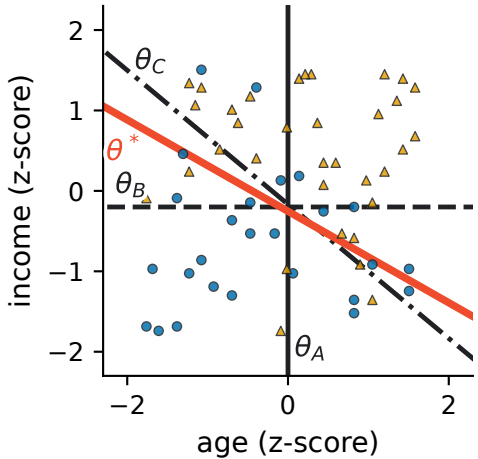
Negative log-likelihood = binary cross-entropy

- Optimizers minimize by convention. Maximizing $\log L$ = minimizing $-\log L$. Dividing by n does not move the minimizer:

$$\underbrace{-\frac{1}{n}\log L(\theta)}_{\text{average } \textbf{negative log-likelihood (NLL)}} = \frac{1}{n}\sum_{i=1}^{n} \underbrace{-\big[\, y_i \log p_i + (1-y_i)\log(1-p_i)\,\big]}_{\ell_{\mathrm{BCE}}(p_i, y_i) = \textbf{binary cross-entropy}} = \widehat{R}(\theta)$$

- The per-sample loss is $\ell_{\mathrm{BCE}} = -\log(\text{probability the model gave to the true label})$.
- ERM with the NLL loss is maximum likelihood estimation. The BCE loss follows from the assumption $y \mid x \sim \mathrm{Bernoulli}$.

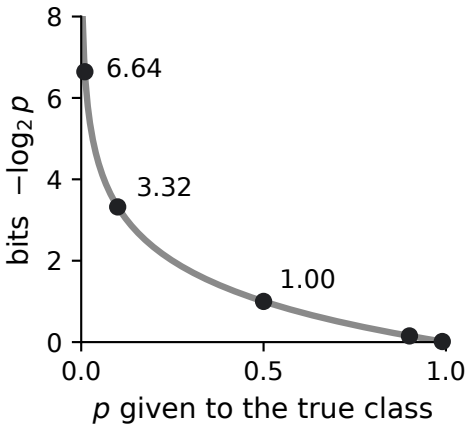
Empirical risk of the three candidate lines



parameters	$\widehat{R}$ (mean BCE)	accuracy
θ_A	0.707	63 %
θ_B	0.526	77 %
θ_C	0.485	73 %
θ^* (red): minimizes $\widehat{R}$	**0.476**	78 %

- All numbers on the 60 training customers. Accuracy = share of correct decisions "buys if $p \geq 0.5$".
- The best θ is defined as the minimizer of $\widehat{R}$.
- Lower loss and higher accuracy need not agree: compare θ_B and θ_C.

Information view: loss as surprise in bits



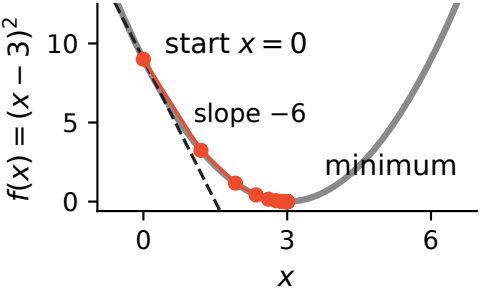
Intuition: $-\log_2 p$ is the **surprise**, in bits, at an outcome to which the model gave probability p.

p (true class)	0.99	0.9	0.5	0.1	0.01
loss in bits	0.014	0.15	1.00	3.32	6.64

- A coin-flip answer costs 1 bit. A confident mistake costs a lot: unbounded as $p \to 0$.
- **Exact**: this is ℓ_{BCE} with $\log_2$ in place of ln (a constant factor).

Note. The average surprise is the **cross-entropy** between the data and the model; for the risk R its lower bound is the **entropy** of the labels given x. Appendix: cross-entropy = entropy + KL divergence.

Gradient descent in one variable: derivative by hand and by autograd



step	x	$f(x)$	by hand	autograd
0	0.000	9.000	-6.000	-6.000
1	1.200	3.240	-3.600	-3.600
2	1.920	1.166	-2.160	-2.160
3	2.352	0.420	-1.296	-1.296

$f(x) = (x-3)^2$, by hand: $f'(x) = 2(x-3)$.
Update: $x \leftarrow x - \eta\, f'(x)$, $\eta = 0.2$, $x_0 = 0$, 25 steps.

```
x = torch.tensor(0.0, requires_grad=True)
f = (x - 3) ** 2
f.backward()                # fills x.grad
x.grad                      # -> -6.000
for step in range(25):
    x.grad = None           # reset
    f = (x - 3) ** 2
    f.backward()
    with torch.no_grad():
        x -= 0.2 * x.grad   # the update
x                           # -> 3.000
```

Dashed: the tangent at the start, slope $f'(0) = -6$. Last two columns: $f'(x)$ by hand and from `backward()`.

Gradient of BCE with sigmoid: $p - y$

The same for the model: the variable is now $\theta = (w, b)$, the function is the loss. One sample: $\ell = -\big[y \log p + (1-y)\log(1-p)\big]$, $p = \sigma(z)$, $z = w^\top x + b$. Two derivatives:

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}, \qquad \frac{\partial p}{\partial z} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = p(1-p)$$

Chain rule, once: $\dfrac{\partial \ell}{\partial z} = \dfrac{\partial \ell}{\partial p} \cdot \dfrac{\partial p}{\partial z} = -y(1-p) + (1-y)p = p - y$

$z = w_1 x_1 + w_2 x_2 + b$ gives $\partial z / \partial w_j = x_j$ and $\partial z / \partial b = 1$, so

$$\frac{\partial \ell}{\partial w_j} = (p-y)\, x_j, \qquad \frac{\partial \ell}{\partial b} = p - y.$$

Intuition: "prediction minus truth", times the input. A buyer ($y = 1$) predicted at $p = 0.2$ gives $p - y = -0.8$: a large correction. A confident correct prediction gives $p - y \approx 0$: almost none.

Gradient descent and one step computed by hand

For parameters $\theta = (\theta_1, \ldots, \theta_m)$ the **gradient** $\nabla_\theta \widehat{R}(\theta) = \left(\partial \widehat{R}/\partial\theta_1, \ldots, \partial \widehat{R}/\partial\theta_m\right)$ is the vector of all partial derivatives. It points in the direction of steepest increase of $\widehat{R}$, so the update steps against it:

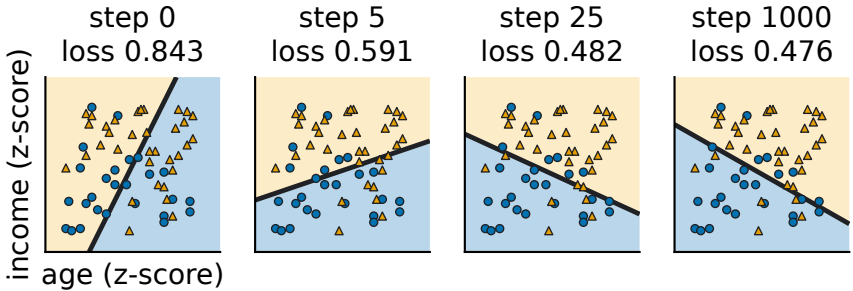
Gradient descent: $\theta \leftarrow \theta - \eta\, \nabla_\theta \widehat{R}(\theta)$, $\eta > 0$ is the **learning rate** (a hyperparameter).

Smallest case: one sample $x = 2$, $y = 1$ (so $\widehat{R} = \ell$), one weight; start $w = -0.5$, $b = 0$; $\eta = 0.5$.

predict	$z = wx + b = -1$,	$p = \sigma(-1)$	$p = 0.2689$, $\ell = -\ln p = 1.313$
gradient	$\partial\ell/\partial w = (p - y)x$,	$\partial\ell/\partial b = p - y$	-1.462, -0.731
update	$w \leftarrow w - \eta\, \partial\ell/\partial w$,	$b \leftarrow b - \eta\, \partial\ell/\partial b$	$w = 0.231$, $b = 0.366$
predict again	$p = \sigma(wx + b)$		$p = 0.696$, $\ell = 0.363$

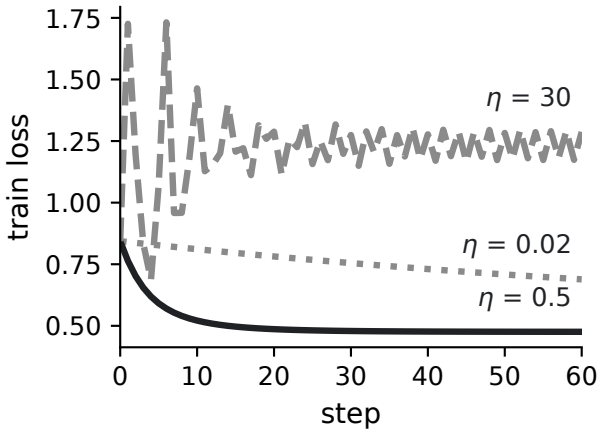
The loss fell from 1.313 to 0.363. **The gradient gives the local downhill direction. It does not give the minimum.**

Gradient descent on the customer data



- The gradient of an average is the average of the gradients: $\nabla_w \widehat{R} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,x_i$. Every unsure or misclassified customer pulls on w, with strength $|p_i - y_i|$ times the size of x_i.
- Full-batch gradient descent on the 60 training customers, $\eta = 0.5$, from a deliberately bad start. Shading: the predicted class.
- After 1000 steps: $\widehat{R} = 0.476$, $w = (0.83, 1.46)$, $b = 0.37$. This is θ^* of the ranking table, up to rounding.

Effect of the learning rate



Same problem, same start (loss 0.84), 60 steps. Only η differs.

- $\eta = 0.02$: right direction, too slow; still at 0.69.
- $\eta = 0.5$: converges, loss 0.48.
- $\eta = 30$: every step overshoots; the loss jumps between about 1.2 and 1.3 and never settles.

The gradient is local information. A step that is too long leaves the region where it is valid.

Distribution assumption, output map and loss

assumption on y given x	network output	output map	loss = NLL
Bernoulli	1 logit z	sigmoid	**BCE**
categorical, K classes	K logits $z_1, \dots, z_K$	softmax	**cross-entropy (CE)**
Gaussian, fixed variance	mean $\hat{y}$	identity	**mean squared error**

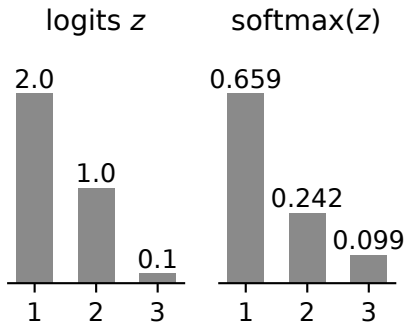
- One procedure: choose $p(y \mid x; \theta) \to$ write the likelihood $\to$ take $-\log \to$ average.
- $f_\theta(x)$ is the raw network output (logits or a mean). The output map turns it into the parameters of $p(y \mid x; \theta)$.

Softmax and cross-entropy for K classes

softmax: $p_k = \dfrac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k = 1, \dots, K$

cross-entropy: $\ell_{\mathrm{CE}}(z, y) = -\log p_y$

Intuition: exponentiate, then normalize.



- $p_k > 0$, $\sum_k p_k = 1$, order kept.
- The label is now a class index $y \in \{1, \dots, K\}$ with $y \mid x \sim \mathrm{Categorical}(p_1, \dots, p_K)$. Product, log, sign, average as before. Example: true class 1 costs $-\ln 0.659 = 0.417$.
- **A language model is this row** with $K =$ vocabulary size (50 257 for GPT-2): next-token prediction is classification with CE.

Note. Computing softmax naively overflows; stable code uses the **log-sum-exp** trick (appendix). For $K = 2$, softmax and CE reduce to sigmoid and BCE (appendix).

MSE as the Gaussian negative log-likelihood

- Regression model: $y \mid x \sim \mathcal{N}(\hat{y}, s^2)$ with mean $\hat{y} = f_\theta(x)$ and a fixed noise variance s^2.
  Density: $p(y \mid x; \theta) = \frac{1}{\sqrt{2\pi s^2}} \exp\Big(- \frac{(y - \hat{y})^2}{2s^2} \Big)$. Take $-\log$:

$$-\log p(y \mid x; \theta) \;=\; \frac{(y - \hat{y})^2}{2s^2} + \underbrace{\tfrac{1}{2} \log(2\pi s^2)}_{\text{does not depend on } \theta}$$

- Minimizing the average NLL over θ is minimizing $\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, the **MSE**.
- **Intuition**: using MSE *is* assuming bell-shaped noise of constant width around the prediction.

PyTorch: calling BCE, CE and MSE

```
z = torch.tensor([2.0, -1.0, 0.5])                  # binary: 3 samples, one LOGIT each
y = torch.tensor([1.0, 0.0, 0.0])                   # target: float 0/1, same shape as z
F.binary_cross_entropy_with_logits(z, y)            # -> 0.471
Z = torch.tensor([[2.0, 1.0, 0.1], [0.2, 0.1, 3.0]])  # K = 3: 2 samples, 3 LOGITS each
t = torch.tensor([0, 2])                            # target: class indices 0..K-1 (int64)
F.cross_entropy(Z, t)                               # -> 0.263
F.cross_entropy(F.softmax(Z, dim=1), t)             # -> 0.711    WRONG, yet no error
y_hat, y_true = torch.tensor([2.5, 0.0]), torch.tensor([3.0, -1.0])
F.mse_loss(y_hat, y_true)                           # -> 0.625
```

- **Logits in, not probabilities.** Both classification losses apply σ or softmax internally, in the numerically stable way. The network therefore ends with a linear layer.
- Passing probabilities raises no error and gives a wrong value (0.711 instead of 0.263): training still runs, only worse.
- **Target format.** BCE: floats 0/1 in the shape of the logits. CE: one integer class index per sample, not a one-hot vector. PyTorch counts classes from 0; the formulas here count from 1.
- By default each function returns the mean over the batch: the empirical risk of the batch.

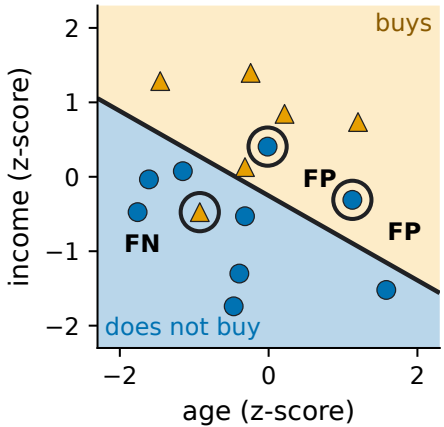
Loss versus metric

Loss
differentiable in θ; the optimizer minimizes it on the training set.
Here: BCE.

Metric
the quantity of practical interest; computed on held-out data; need not be differentiable.
Here: errors and what they cost.

- The loss is optimized because it has a gradient. The metric is reported because it is what matters.
- The model outputs a probability p. A decision needs a **decision threshold** t: predict "buys" if $p \geq t$.
- All evaluation numbers are measured on the 15 test customers, which the model has never seen.

Test set of 15 customers: confusion matrix



	predicted: buys	predicted: not
actual: ▲ 1 (6 buyers)	**5** TP	**1** FN
actual: ● 0 (9 non-buyers)	**2** FP	**7** TN

Confusion matrix: counts of (actual class, predicted class) at threshold $t = 0.5$. "Positive" = buys.
TP true positives, FN false negatives,
FP false positives, TN true negatives.

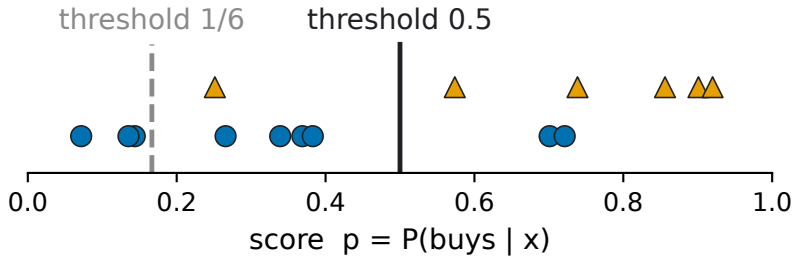
Map: the 15 test customers and the trained boundary $p = 0.5$. Ringed: the three errors, 2 FP above the line and 1 FN below.

Precision, recall (TPR), FPR and accuracy

precision	$= \frac{TP}{TP+FP} = \frac{5}{5+2} = 0.714$	share of the predicted buyers who really buy
recall = TPR	$= \frac{TP}{TP+FN} = \frac{5}{5+1} = 0.833$	share of the real buyers that were found (true positive rate)
FPR	$= \frac{FP}{FP+TN} = \frac{2}{2+7} = 0.222$	share of the non-buyers that were flagged (false positive rate)
accuracy	$= \frac{TP+TN}{\text{all}} = \frac{5+7}{15} = 0.800$	share of correct decisions

- TPR and FPR are the two coordinates of the ROC curve.

Decision threshold and misclassification costs

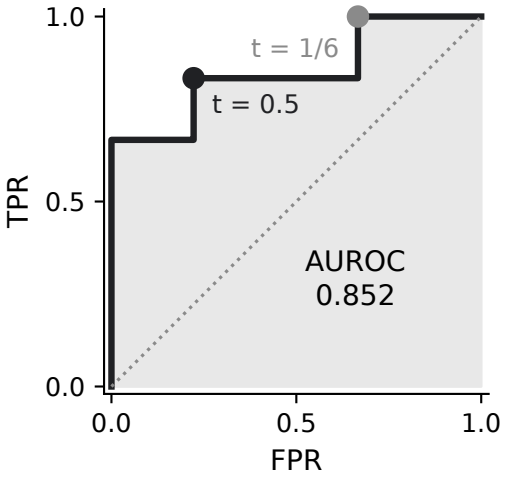


Each marker is one test customer at its score p; upper row: buyers, lower row: non-buyers.

- FP = a discount sent to a non-buyer (cost 1). FN = a missed buyer (cost 5).
- Send when not sending costs more in expectation: $5p \geq 1 \cdot (1-p) \iff p \geq \frac{1}{6}$; in general $t = \frac{c_{FP}}{c_{FP}+c_{FN}}$, if p can be trusted as a probability.

threshold	TP	FN	FP	TN	recall	total cost $= 1 \cdot FP + 5 \cdot FN$
$t = 0.5$	5	1	2	7	0.833	7
$t = 1/6$	6	0	6	3	1.000	6

ROC curve

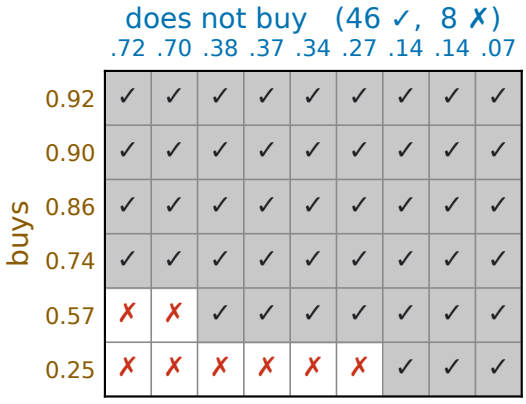


Intuition: the threshold sweep

- Move t from 1 down to 0 along the score line. At each t plot (FPR, TPR).
- t high: nobody flagged, $(0, 0)$. t low: everybody flagged, $(1, 1)$.
- Passing a buyer moves the curve up by $\frac{1}{6}$. Passing a non-buyer moves it right by $\frac{1}{9}$.

ROC (receiver operating characteristic) **curve**: the points $\big(\mathrm{FPR}(t), \mathrm{TPR}(t)\big)$ for all thresholds t. **AUROC**: the area under it. Random scores give about 0.5 (the diagonal); a perfect ranking gives 1.

AUROC as a pair-counting probability



Rows: the 6 buyers' scores. Columns: the 9 non-buyers' scores.

✓ = the buyer has the higher score.

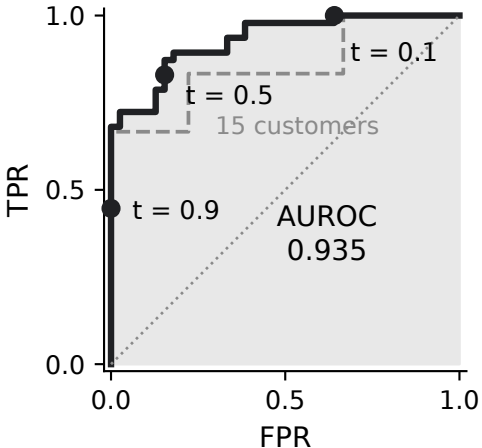
Exact

$\mathrm{AUROC} = P\big(s(x^+) > s(x^-)\big)$ for a random positive x^+ and a random negative x^-; s is the model's score. Ties count $\frac{1}{2}$.

Estimate: $\dfrac{\#\{(i,j) \,:\, s_i^+ > s_j^-\}}{n_+\, n_-}$

- $6 \times 9 = 54$ pairs; 8 are in the wrong order, 46 are correct: $\mathrm{AUROC} = 46/54 = 0.852$.
- Each grey cell is one rectangle under the ROC staircase. AUROC measures the *ranking*; no threshold needed.

ROC curve of the penguin model



- The model of the 1D example (male from body mass), evaluated on the 86 held-out penguins: 47 males, 39 females.
- $\mathrm{AUROC} = 0.935$, over $47 \times 39 = 1\,833$ pairs. Dashed: the ROC curve of the 15 test customers, AUROC 0.852.
- A threshold on p is a threshold on body mass:

threshold t	0.9	0.5	0.1
body mass (kg)	4.1	3.7	3.3
TPR	0.45	0.83	1.00
FPR	0.00	0.15	0.64

Accuracy at $t = 0.5$: 84 % on the held-out birds, 84 % on the 128 birds used for fitting.

scikit-learn: confusion matrix, precision, recall, AUROC

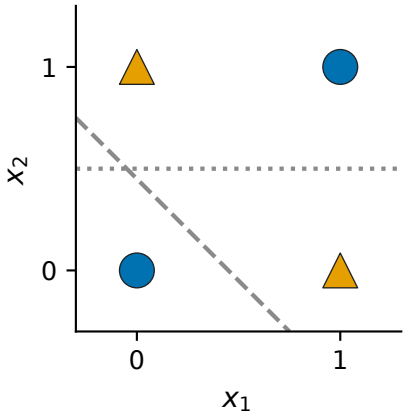
```
from sklearn.metrics import (confusion_matrix, precision_score,
                             recall_score, roc_auc_score)
y_pred = (p >= 0.5).astype(int)                    # p, y: scores, labels (15 customers)
confusion_matrix(y, y_pred)                        # -> [[7, 2], [1, 5]]
precision_score(y, y_pred)                         # -> 0.714
recall_score(y, y_pred)                            # -> 0.833
roc_auc_score(y, p)                                # -> 0.852
```

- scikit-learn orders the confusion matrix by class value, rows = actual, columns = predicted: $\big[[TN, FP], [FN, TP]\big]$. The 2×2 table of the customer example lists the positive class first.
- Precision and recall need decisions (`y_pred`). AUROC needs the scores (`p`); decisions would discard the ranking.

Evaluation caveats: class imbalance, calibration, variance over runs

- **Class imbalance.** With 1 % buyers, "nobody buys" reaches 99 % accuracy and recall 0. Use precision and recall, summarized over all thresholds by the area under the precision-recall curve (**PR-AUC**).
- **Calibration.** σ always outputs a number in $(0, 1)$. It is calibrated if, among customers with $p \approx 0.8$, about 80 % buy. A model can rank well and still be badly calibrated.
- **One run is not evidence.** Initialization is random, and 15 test customers are few. Report mean $\pm$ sample standard deviation over several seeds.

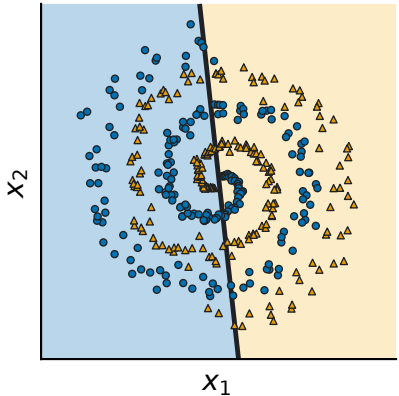
XOR is not linearly separable



- Four points; class ▲ 1 if exactly one coordinate is 1 (**XOR**, "exclusive or").
- No line puts both triangles on one side and both circles on the other; the two grey lines are failed attempts.
- Minsky and Papert (*Perceptrons*, 1969) proved limits of this kind for single-layer perceptrons; XOR is the best-known case. Interest in neural networks then dropped for years.
- The problem is not the training. The **model family** contains no solution.

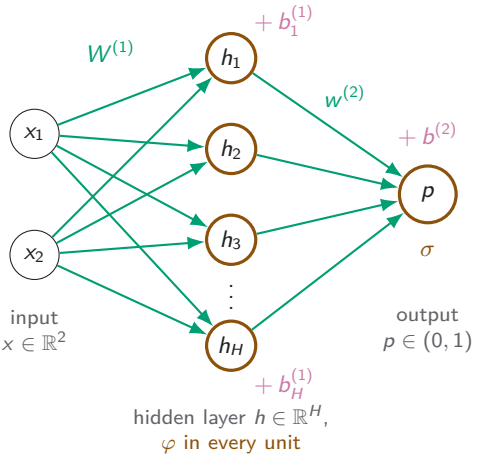
A hand-made feature can make such data linearly separable (**feature engineering**; an example is in the appendix), but the feature has to be designed by a person.

Spiral data: logistic regression at chance level



- Synthetic data: two classes on two interleaved spiral arms with noise. 400 training points and 400 test points from the same generator; the test points are shown.
- Logistic regression, trained to convergence: test accuracy 52.0 %. A coin flip gets 50 %. Shading: predicted class; line: $p = 0.5$.
- A feature that separates the two arms is hard to design by hand.
- Instead, the model can learn the features from the data, with the same gradient descent as its other parameters.

Multilayer perceptron (MLP): architecture and equations



$h = \varphi(W^{(1)} x + b^{(1)})$ hidden layer

$p = \sigma(w^{(2)\top} h + b^{(2)})$ output

$x \in \mathbb{R}^2$, $W^{(1)} \in \mathbb{R}^{H \times 2}$, $b^{(1)} \in \mathbb{R}^H$, $h \in \mathbb{R}^H$, $w^{(2)} \in \mathbb{R}^H$, $b^{(2)} \in \mathbb{R}$. The superscript (l) numbers the layer; a subscript always picks an entry.

- φ: a nonlinear **activation function** (e.g. ReLU), applied to each entry.
- One hidden unit = one row of $W^{(1)}$ (one line in the plane), then φ.
- Output layer: a logistic regression on the learned features h instead of x.
- $\theta = (W^{(1)}, b^{(1)}, w^{(2)}, b^{(2)})$; the width H is a hyperparameter.

Linear layer as an affine map: worked example

$W \in \mathbb{R}^{3\times 2}$, $x \in \mathbb{R}^2$, $b \in \mathbb{R}^3$: a vector with 2 entries goes in, a vector with 3 entries comes out.

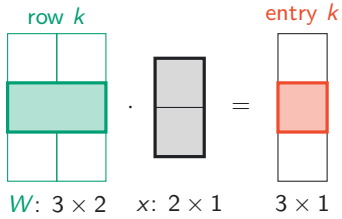
$$Wx + b = \underbrace{\begin{pmatrix} 1 & 2 \\ -1 & 3 \\ 2 & -2 \end{pmatrix}}_{3\times 2} \underbrace{\begin{pmatrix} 2 \\ 1 \end{pmatrix}}_{2\times 1} + \underbrace{\begin{pmatrix} 1 \\ 5 \\ -2 \end{pmatrix}}_{3\times 1} = \begin{pmatrix} 1\cdot 2 + 2\cdot 1 + 1 \\ (-1)\cdot 2 + 3\cdot 1 + 5 \\ 2\cdot 2 + (-2)\cdot 1 + (-2) \end{pmatrix} = \underbrace{\begin{pmatrix} 5 \\ ? \\ 0 \end{pmatrix}}_{3\times 1}$$

The entry marked ? is computed on the next step.

Answer: take row 2 of W, multiply it with x entry by entry, add up, then add entry 2 of b: $(-1)\cdot 2 + 3\cdot 1 + 5 = 6$. **In general**: entry k of the result is $\sum_j W_{kj}\, x_j + b_k$: row k of W times x, plus b_k. Each output entry is one hidden unit before its activation: a dot product plus a bias.

Written in the style of Definition 1.1.1 (affine functions) in Jentzen, Kuckuck, von Wurstemberger, arXiv:2310.20360.

Matrix-vector product: reminder



Row k times the column gives entry k.

Note: matrix-vector product.

- **Row times column.** $(Wx)_k = \sum_j W_{kj}\, x_j$.
- **Shape rule.** $(m \times n)(n \times 1) = (m \times 1)$: inner sizes agree, outer sizes give the result.
- **Mixing.** Every output entry mixes *all* input entries. So a layer can map $\mathbb{R}^{784} \to \mathbb{R}^{10}$; entry-by-entry multiplication cannot change the number of entries.
- **Linear vs. affine.** $x \mapsto Wx$ is **linear** (the origin stays fixed). $x \mapsto Wx + b$ is **affine**. A "linear layer" is the affine map.

PyTorch: nn.Linear and tensor shapes

```
layer = nn.Linear(in_features=2, out_features=3)   # W: 3 x 2, b: 3 entries
layer.weight.shape, layer.bias.shape               # -> (3, 2), (3,)
with torch.no_grad():                              # put in W and b of the previous slide
    layer.weight.copy_(torch.tensor([[1., 2.], [-1., 3.], [2., -2.]]))
    layer.bias.copy_(torch.tensor([1., 5., -2.]))
x = torch.tensor([2., 1.])                         # one input vector
layer(x)                                           # -> [5, 6, 0]
X = torch.randn(5, 2)                              # a batch: 5 input vectors, one per ROW
layer(X).shape                                     # -> (5, 3)
```

- `weight` has the shape (out, in), the same layout as W in $Wx + b$; with the values of the worked example the layer returns $(5, 6, 0)$.
- Input shape (B, in), output shape (B, out): B samples are processed at once, one per row.
- This layer has $3 \cdot 2 + 3 = 9$ parameters, created with random values and changed only by training.

Convention. Mathematical notation: column vectors, $Wx + b$. PyTorch: a batch of *rows* $X \in \mathbb{R}^{B \times \text{in}}$ and $XW^\top + b$, with b added to each row. Same numbers, transposed layout.

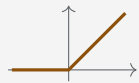
Composition of affine maps is affine; ReLU

- Remove φ and stack two layers:

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = \underbrace{(W^{(2)}W^{(1)})}_{\text{one matrix}}\, x + \underbrace{(W^{(2)}b^{(1)} + b^{(2)})}_{\text{one vector}}.$$

  A composition of affine maps is affine: the boundary stays a line, however many layers. (In the network above $W^{(2)} = w^{(2)\top}$, a $1 \times H$ matrix.)

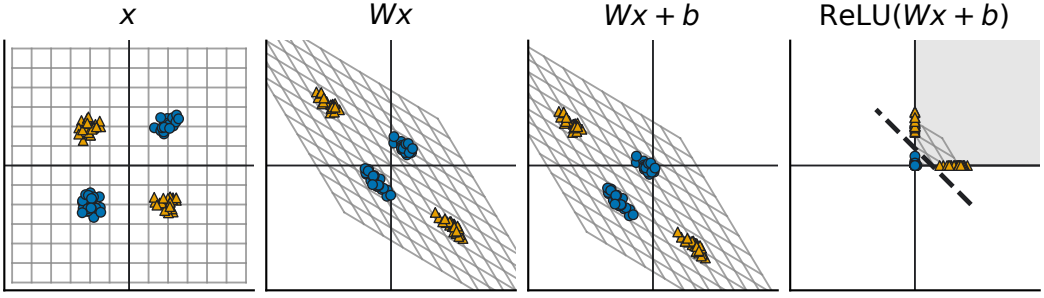
ReLU (rectified linear unit): $\mathrm{ReLU}(z) = \max(0, z)$, applied to each entry.



- ReLU is piecewise linear, not linear. With it between the layers the composition is no longer one affine map: each hidden unit adds one bend along one line.
- Standard choice for binary classification: ReLU in the hidden layers, σ at the output.

ReLU as in Definition 1.2.4 of Jentzen et al., arXiv:2310.20360.

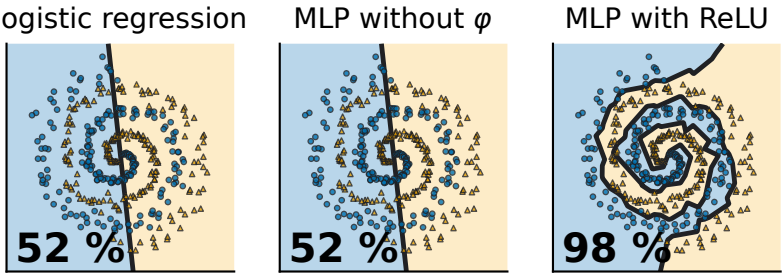
Geometry of one layer: affine map and ReLU



A layer moves *all* points of the plane at once. Grey: a square grid. Clouds: XOR data with coordinates ± 1.

- Wx: this W stretches the grid along one diagonal and squeezes it along the other (in general W can also rotate and shear). Lines stay lines, the origin stays put. Then $+\, b$ shifts everything.
- ReLU sets negative entries to 0: the shaded quadrant is left unchanged, everything else is flattened onto its edges. The new axes are the two hidden units h_1, h_2 (rows 1 and 2 of W).
- After that, the dashed line $h_1 + h_2 = 0.44$ separates XOR. Before, no line could.

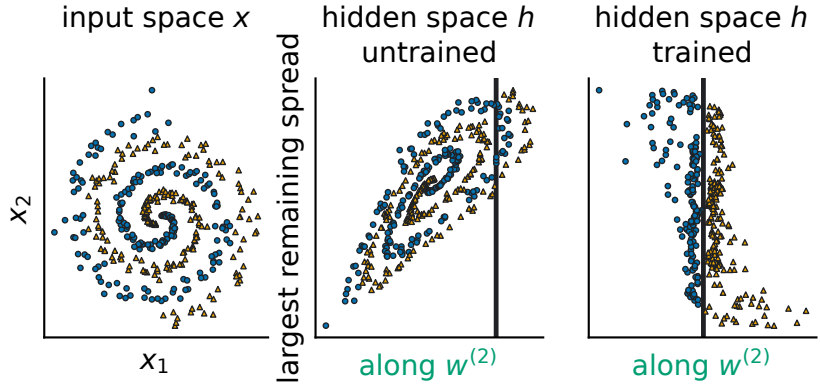
Spiral: logistic regression, linear MLP, ReLU MLP



Large numbers: test accuracy. Shading: predicted class. Points: the 400 test points.

- Same data, same training code: Adam, learning rate 0.01, 1500 epochs, full batch.
- $2 \to 64 \to 1$ without φ: 257 parameters, yet the same straight boundary as logistic regression (3 parameters).
- With ReLU: 98.3 % test accuracy; over 5 initializations 98.0 % $\pm$ 0.5 %.
- Parameters of $2 \to 64 \to 1$: $64 \cdot 2 + 64 + 64 + 1 = 257$.

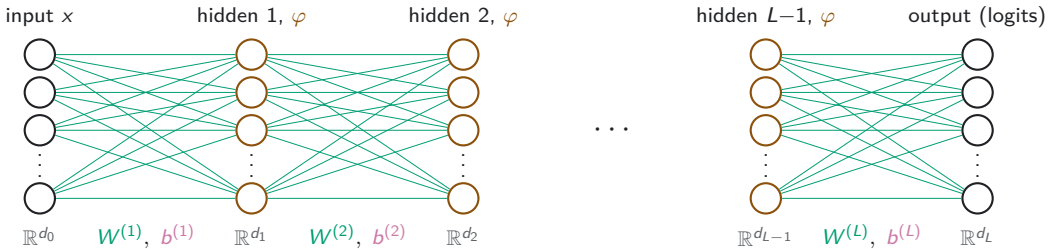
Hidden representation of the trained MLP



The 400 training points; their hidden vectors $h \in \mathbb{R}^{64}$ projected on $w^{(2)}$ and on the perpendicular direction of largest variance. Vertical line: the boundary $w^{(2)\top} h + b^{(2)} = 0$.

- Untrained: mixed. Trained: one hyperplane separates 100 % of the training and 98.3 % of the test points.
- **Representation learning**: the last layer is logistic regression on learned features h.

General MLP with L layers



$$h^{(0)} = x, \qquad h^{(l)} = \varphi\big(W^{(l)}\, h^{(l-1)} + b^{(l)}\big) \text{ for } l = 1, \dots, L-1, \qquad f_\theta(x) = W^{(L)}\, h^{(L-1)} + b^{(L)},$$
$$W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}, \quad b^{(l)} \in \mathbb{R}^{d_l}.$$

- An MLP alternates affine maps and activations. The spiral network is the case $L = 2$, $d_0 = 2$, $d_1 = 64$, $d_2 = 1$.
- "Deep": several such layers composed. No fixed number of layers makes a network deep.

Diagram in the spirit of Figures 1.1 and 1.2 in Jentzen et al., arXiv:2310.20360.

The training loop in PyTorch

```
model = nn.Sequential(nn.Linear(2, 64), nn.ReLU(),
                      nn.Linear(64, 1))          # 1
loss_fn = bce                                    # 2
optimizer = torch.optim.Adam(model.parameters(), lr=lr)

for epoch in range(epochs):                      # 3
    order = torch.randperm(len(X), generator=g)
    for idx in order.split(batch_size or len(X)):  # 4
        loss = loss_fn(model(X[idx]), y[idx])    # 5
        optimizer.zero_grad()                    # 6
        loss.backward()                          # 7
        optimizer.step()                         # 8
```

1. f_θ: $2 \to 64 \to 1$. It returns the *logit* z.
2. BCE from the logit: σ is applied inside the loss, the stable way.
3. epoch: one pass over the training set.
4. batch: a subset; one batch, one step.
5. empirical risk of the batch.
6. clear old gradients.
7. gradient of the batch loss.
8. $\theta \leftarrow \theta - \mathtt{lr} \cdot (\dots)$.

`X`, `y`: training data. `g`: a seeded random generator. Spiral: `lr = 0.01`, `epochs = 1500`, full batch.

Line 7 computes the gradient automatically; line 8 applies Adam, an update rule of the gradient-descent type.

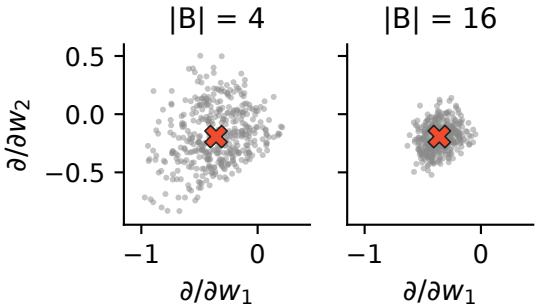
Batch loss as an unbiased estimate of the empirical risk

For a batch B of indices drawn uniformly at random from $\{1, \dots, n\}$:

$$\widehat{R}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \ell\big(f_\theta(x_i), y_i\big)$$

$$\mathbb{E}_B\Big[\widehat{R}_B(\theta)\Big] = \widehat{R}(\theta), \quad \mathbb{E}_B\Big[\nabla_\theta \widehat{R}_B(\theta)\Big] = \nabla_\theta \widehat{R}(\theta)$$

Every sample enters the batch with the same probability, and expectation and gradient are linear. So batch loss and batch gradient are **unbiased estimates**: right on average, noisy in one draw, less so for larger $|B|$.



Customers, at θ_0. Grey: ∇_w of 400 random batches. Cross: the full gradient $(-0.357, -0.192)$. Mean of 2000 batches of 4: $(-0.356, -0.185)$.

- **Why batches.** A step costs $|B|$ samples, not n: Fashion-MNIST with $|B| = 128$ makes 430 updates per pass over its 55 000 images instead of one. A batch fits into memory and runs in parallel. Full batch (customers, spiral) gives the exact gradient.

Glossary: sample, batch, step, epoch, learning rate, optimizer

sample	one pair (x_i, y_i)
batch	a subset of the training set used for one gradient computation; its size is a hyperparameter
step	one parameter update, computed from one batch
epoch	one pass over the whole training set $= \lceil n/\text{batch size} \rceil$ steps
learning rate	the factor η that scales every update
optimizer	the rule that turns gradients into updates (SGD, Adam, ...)

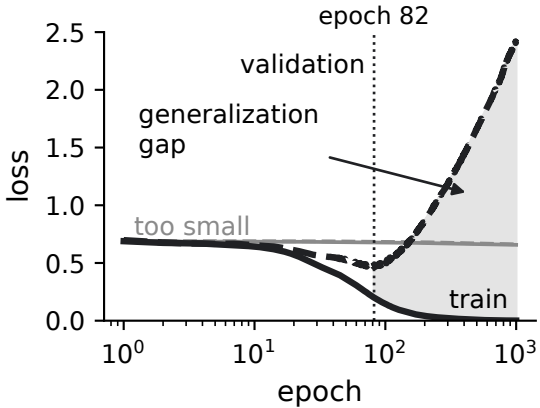
Gradient descent with batch gradients, which are unbiased estimates of $\nabla_\theta \widehat{R}$, is **stochastic gradient descent** (SGD).

Training, validation and test sets

training set	validation set	test set
fits the parameters θ (and the scaling statistics)	chooses hyperparameters and when to stop	estimates $R(\theta)$; used once, at the end

- $\widehat{R}$ on the training set is optimistic: θ was chosen to make it small. Held-out data estimate R honestly.
- Every decision made by looking at a set makes that set optimistic too. Hence validation data for decisions, and an untouched test set for the final number.
- Customers, mean BCE: training 0.476; the 15 test customers 0.474 (a noisy estimate from 15 points). Because the data generator is known, R can be estimated on 200 000 fresh customers: $R \approx 0.49$. The gap $R - \widehat{R} \approx 0.01$ is small; roughly, 3 parameters cannot memorize 60 points.

Underfitting and overfitting: loss curves



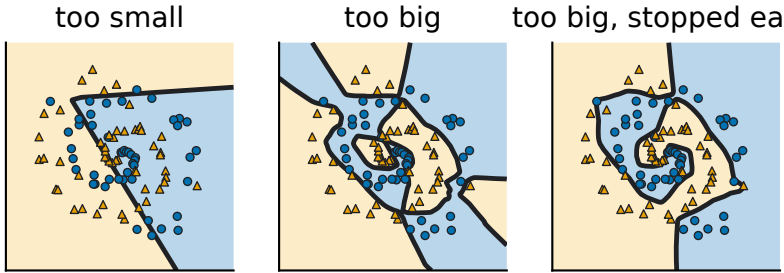
Solid: training, dashed: validation. Black: big network, grey: small. Dotted: epoch of lowest validation loss. Data: a noisier spiral, 100 training and 400 validation points, each label flipped with probability 10 %.

Underfitting: $\widehat{R}(\theta)$ stays large; the family cannot fit even the training data.

Overfitting: $\widehat{R}(\theta)$ is small but the generalization gap $R(\theta) - \widehat{R}(\theta)$ is large. Estimate of the gap: validation loss minus training loss.

- Too small, $2 \to 2 \to 1$ (9 parameters): both losses stay near 0.66.
- Too big, $2 \to 256 \to 256 \to 1$ (66 817 parameters): training loss $\to 0.0008$, validation loss climbs to 2.41. It fits even the flipped labels; on new points it is wrong with high confidence.

Underfitting and overfitting: decision boundaries



Points: the 100 training points. Shading: predicted class. Adam, learning rate 0.003, 1000 epochs, full batch. No test set is used: no final number is claimed.

- Too small: a boundary with at most two bends. Training 52 %, validation 55.5 %.
- Too big: an irregular boundary that bends around single noisy points. Training 100 %, validation 80.5 %.
- **Early stopping**: keep the parameters from the minimum of the validation curve, epoch 82 (validation loss 0.47): training 94 %, validation 85.3 %. Other remedies, by name: more or better data, a smaller model, regularization, augmentation.

Fashion-MNIST: MLP 784-256-10 with cross-entropy



- 70 000 grey-scale images of clothing, 28×28 pixels, 10 classes. An image is a vector $x \in \mathbb{R}^{784}$.
- 55 000 training, 5 000 validation (used to choose the number of epochs), 10 000 test images.
- MLP $784 \to 256 \to 10$ with ReLU: 203 530 parameters, 10 logits. Loss: softmax + cross-entropy.
- The same loop: Adam, learning rate 0.001, 5 epochs, batches of 128.

validation accuracy 88.1 %

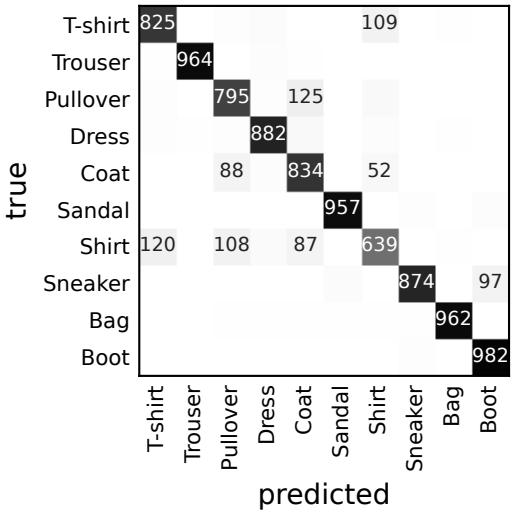
test accuracy 87.1 %

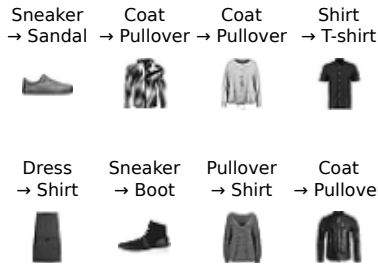
over 5 seeds: 87.0 % ± 0.2 %

Random guessing: 10 %. Training takes seconds on a laptop CPU.

The spiral model and the clothing model are trained the same way: a parametric function, an NLL loss, gradient steps, held-out evaluation.

Fashion-MNIST: confusion matrix and misclassified examples

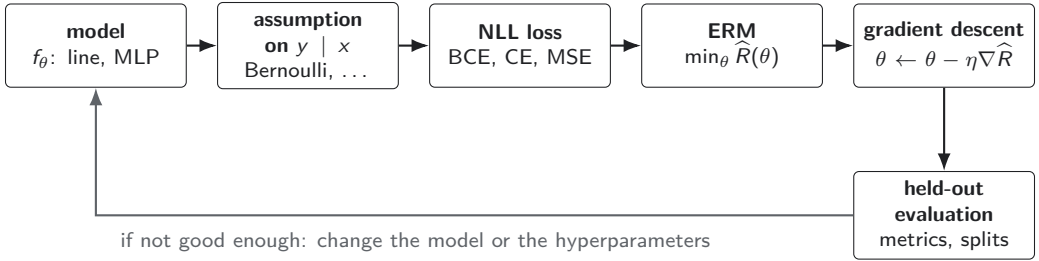




The first 8 test errors: true → predicted.

- The 10×10 **confusion matrix** of the test set, 1000 images per row. Printed: the diagonal and confusions made ≥ 50 times.
- Errors sit on upper-body garments; most frequent: Pullover → Coat (125).

Summary: model, loss, optimization, evaluation



Deep learning = a parametric function f_θ + a loss derived from a probabilistic model + minimization of empirical risk by gradient methods + evaluation on data the model has not seen.

For one layer the gradient was computed by hand: $(p - y)\, x$. The Fashion-MNIST network has 203 530 parameters, current language models have billions; there the gradient is computed automatically.

Key terms

Model

- feature vector, label
- parameters, hyperparameters
- model family $\{f_\theta\}$
- logit, sigmoid, softmax
- affine map, activation, ReLU
- hidden representation

Loss and optimization

- i.i.d., $p(y \mid x; \theta)$
- likelihood, log-likelihood, NLL, MLE
- BCE, CE, MSE
- risk, empirical risk, ERM
- gradient, learning rate
- batch, step, epoch, optimizer

Evaluation

- loss vs. metric, threshold
- confusion matrix
- precision, recall, TPR, FPR
- ROC, AUROC
- train / validation / test
- generalization gap, underfitting, overfitting

Further reading

- **Rigorous treatment**: A. Jentzen, B. Kuckuck, P. von Wurstemberger, *Mathematical Introduction to Deep Learning: Methods, Implementations, and Theory*, arXiv:2310.20360. The Introduction and Chapter 1 define affine functions, fully-connected networks and activation functions rigorously.
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press 2016, Chapters 5 and 6. `deeplearningbook.org`
- C. M. Bishop, H. Bishop, *Deep Learning: Foundations and Concepts*, Springer 2024, Chapters 4 to 6.
- T. Fawcett, *An introduction to ROC analysis*, Pattern Recognition Letters 27 (2006).

Appendix: notation

symbol	meaning
$x \in \mathbb{R}^d$, y, $\hat{y}$	features, label, prediction
$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$	dataset, n samples
x_j, μ_j, s_j	feature j: mean, std (train)
P	data distribution
f_θ, θ, Θ	model, parameters, their set
θ^*, θ_{MLE}	$\arg\min \widehat{R}$, $\arg\max L$
W, w / b	weights / biases
$W^{(l)}$, $b^{(l)}$, $h^{(l)}$	of layer l; hidden vector
W_{kj}, b_k, w_j	entries
z, z_k	logit(s)
σ, φ	sigmoid, activation

symbol	meaning
p, p_i, p_k	predicted probability
ℓ; R, $\widehat{R}$	loss; risk, empirical risk
$L(\theta)$	likelihood
∇_θ, η	gradient, learning rate
t; $s(x)$	threshold; ranking score
TP, FP, FN, TN	confusion counts
n_+, n_-	positives, negatives
H; d_l; L (in MLP)	widths; depth
K	number of classes
$\mathrm{H}(\cdot)$, $\mathrm{H}(\cdot,\cdot)$, D_{KL}	entropy, cross-entropy, KL
s^2	noise variance

Vectors are columns. log is the natural logarithm unless written $\log_2$. L denotes the likelihood in the context of losses and the depth in the general MLP. A hat means "computed from data": $\widehat{R}$ from the training set, $\hat{y}$ from the model.

Appendix: cross-entropy, entropy and KL divergence

Note. For a data distribution p_{data} and a model distribution p_θ over the same outcomes k:

$$\underbrace{\mathrm{H}(p_{\text{data}}, p_\theta)}_{\text{cross-entropy}} = -\sum_k p_{\text{data},k} \log p_{\theta,k} \;=\; \underbrace{\mathrm{H}(p_{\text{data}})}_{\text{entropy}} + \underbrace{D_{\mathrm{KL}}(p_{\text{data}} \,\|\, p_\theta)}_{\text{KL divergence} \geq 0},$$

with $\mathrm{H}(p) = -\sum_k p_k \log p_k$ and $D_{\mathrm{KL}}(p \,\|\, q) = \sum_k p_k \log(p_k / q_k)$. (Split the logarithm.)

- The entropy does not depend on θ. So minimizing the cross-entropy is minimizing the KL divergence from the data distribution to the model; $D_{\mathrm{KL}} = 0$ only if the two are equal.
- For the risk R, p_{data} is the true conditional distribution of y given x. With noisy labels its entropy is positive, so $R(\theta)$ cannot reach 0.
- For the empirical risk, p_{data} is the one-hot label of each training sample, whose entropy is 0: $\widehat{R}(\theta) = \frac{1}{n} \sum_i \mathrm{H}(\text{one-hot}(y_i),\, p(\cdot \mid x_i; \theta))$ can be driven towards 0 by memorizing (if the network is large enough and no two identical inputs carry different labels). The overfitting experiment (training loss 0.0008) is an instance.

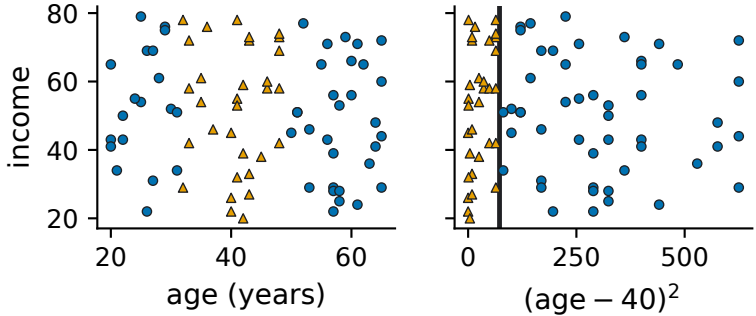
Appendix: log-sum-exp, and two notes on the logistic model

Note: the log-sum-exp trick. $\log p_k = z_k - \log \sum_j e^{z_j}$. With $m = \max_j z_j$: $\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$. Every exponent is now ≤ 0, so nothing overflows, and the result is identical (shift invariance). Library losses use it, and a hand-written loss needs it as well. For $K = 2$: $p_1 = \sigma(z_1 - z_2)$, and CE becomes BCE with $z = z_1 - z_2$.

Note: why not sigmoid with squared error. With $\ell = \frac{1}{2}(p - y)^2$ and $p = \sigma(z)$: $\partial \ell / \partial z = (p - y)\, p(1 - p)$. For a confidently wrong prediction ($p \approx 0$, $y = 1$) the factor $p(1 - p) \approx 0$ removes the gradient exactly when it is needed. With BCE the factor cancels: $\partial \ell / \partial z = p - y$.

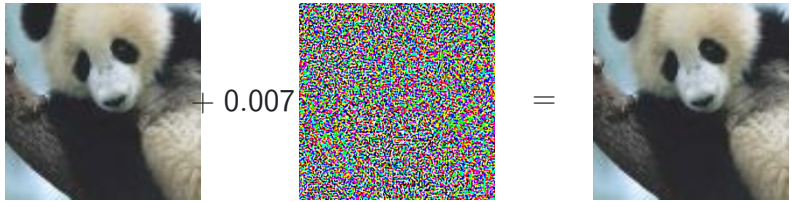
Note: linearly separable data have no maximum-likelihood estimate. Take any (w, b) that separates the training classes. Multiplying it by $c > 1$ pushes every p_i towards its label and lowers the loss. The infimum 0 is approached only as $\|w\| \to \infty$; no finite minimizer exists, and gradient descent increases $\|w\|$ forever. One reason for regularization and early stopping.

Appendix: feature engineering example



- Another synthetic shop, 80 customers: the buyers are middle-aged. Logistic regression gets 46 % training accuracy; always saying "no" gets 61 % (minimizing BCE is not maximizing accuracy).
- With the hand-made feature $(\text{age} - 40)^2$ one vertical line separates them (100 %). This is **feature engineering**.

Appendix: adversarial example



"panda", 57.7 % confidence a computed perturbation "gibbon", 99.3 % confidence

This course is about understanding why these systems work and why they fail.

Figure and numbers: Goodfellow, Shlens, Szegedy, *Explaining and Harnessing Adversarial Examples*, ICLR 2015, arXiv:1412.6572, Fig. 1.

Appendix: sources and licences of third-party material

- Image classification: ConvNeXt-Tiny (torchvision, ImageNet weights); photo: Uoaei1, Wikimedia Commons, CC BY-SA 4.0. Detection and masks: Mask R-CNN (torchvision, COCO weights); photo: Jos1950, Wikimedia Commons, CC BY-SA 4.0.
- Next-token distribution: computed with GPT-2 small (Radford et al., 2019; weights under the MIT licence, Hugging Face `openai-community/gpt2`).
- Protein structure: AlphaFold Protein Structure Database, entry AF-P69905-F1 (model v6), CC BY 4.0. Jumper et al., Nature 596 (2021); Varadi et al., Nucleic Acids Research 52 (2024).
- Speech sample: VOiCES corpus, CC BY 4.0. Richey et al., Interspeech 2018.
- Adversarial example: Goodfellow, Shlens, Szegedy, ICLR 2015, arXiv:1412.6572, Figure 1. Shown as a cited quotation; not openly licensed.
- Fashion-MNIST: Xiao, Rasul, Vollgraf, arXiv:1708.07747, MIT licence.
- Definitions and diagrams adapted from Jentzen, Kuckuck, von Wurstemberger, arXiv:2310.20360, where marked.

All other figures are generated by the scripts in `build/` with fixed seeds. Customer and spiral data are synthetic.